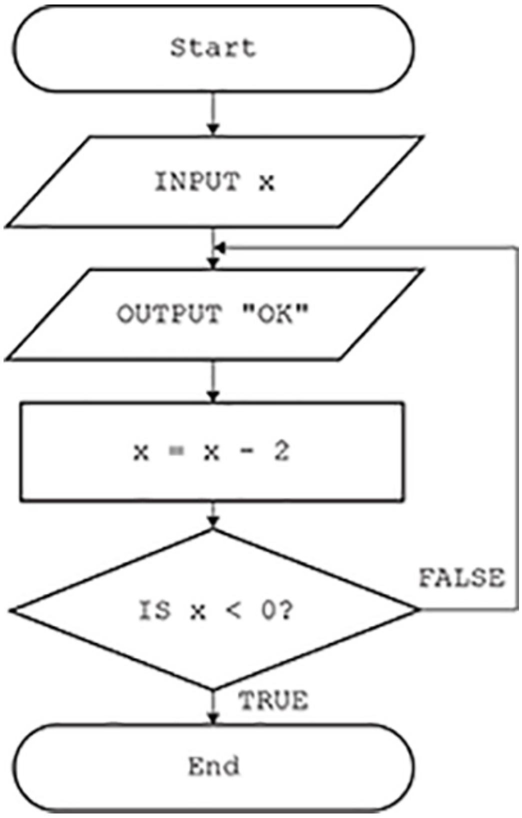
1 This flowchart uses iteration.



A programmer implements the flowchart as a program in a high-level language using an Integrated Development Environment (IDE).

Identify and describe **two** tools that the programmer could use in the IDE to create a program from the flowchart.

Tool 1 ______________________________

Description ______________________________

Tool 2 ______________________________

Description ______________________________ **[4]**

2(a) A programmer uses a high-level language and an Integrated Development Environment (IDE) to create a computer game.

Describe **two** advantages to the programmer of using a high-level language instead of a low-level language.

1 ____

____

____

____

2 ____

____

____

____ **[4]**

(b) The IDE includes a translator, such as a compiler or an interpreter.

Tick (✓) **one** box in each row to identify whether each statement describes the use of a compiler, an interpreter, or both.

Statement	Compiler	Interpreter	Both
Translates high-level code to low-level instructions.			
Produces an executable file.			
Program needs to be translated every time it is run.			

[3]

3 Victoria creates a program using an Integrated Development Environment (IDE).

Describe two tools or facilities that an IDE commonly provides.

-- [4]

END OF QUESTION PAPER

Question			Answer/Indicative content	Marks	Guidance
1			1 mark for tool 1 mark for matching description e.g. • Text/code editor… • …allows program code to be **written** / **entered / changed** • …allows errors to be fixed • Debugger / error reporting / diagnostics… • …identifies / finds / checks for errors • …shows location / detail of errors • …suggests fixes • Run-time environment / output window… • …allows **program / code** to be run / executed • …shows output of the **program / code** • Translator / compiler / interpreter … • … convert to **low-level/machine code** • …allow program to be executed / run • …produce executable file (only for compiler) • …stops execution when error found (interpreter only) • Stepping … • … execute/run the program line by line • Variable watch… • … see the **contents/data** held in variables / see how (contents of) variables **change** • Break points … • … will allow the program to stop **at a chosen / set position** • Pretty printing // keyword highlighting… • … allows keywords / variables to be coloured / identified • Keyword completion // autocomplete // autocorrect // syntax suggestion… • …suggests/corrects code/syntax (when first part entered). • GUI (builder)… • …allows creation of user interface / buttons / by example	4	Allow other sensible names for features. Description must add more than is given in the identification of the feature to be awarded. For example, “keyword highlighting, highlights keywords” is 1 mark for the feature only. If compiler and interpreter given as two distinct features, allow both (with suitable descriptions). Do not allow translator and compiler/interpreter. **Description must match tool. If tool name wrong / missing, do not give description.** Allow sensible references to AI where appropriate. Allow other sensible features of an IDE (e.g. line numbering, **auto** indent, collapsed blocks, etc) with suitable description. Description must add (slightly) more than just repeating the feature name. Do not allow features of programming languages (comments, casting, etc). For text editor / error diagnostics / debugger, allow other sensible features listed as features in the mark scheme as description (e.g. “text editor does pretty printing”, “debugger does stepping”) <u>Examiner’s Comments</u> Candidates were mostly able to name and describe suitable tools contained within an Integrated Development Environment, such as an editor, error diagnostics, run-time environment or translator. Many other sensible tools and sub-tools such as a variable watch window, stepping, keyword completion or Artificial Intelligence (AI) tools were all also credited. A small number of candidates mistakenly described tools contained within programming languages, such as commenting or use of variables. Descriptions were generously given where they added to the tool name, but repetition (such as “a code editor / edits code”) was

<table>
<tr><th colspan="3">Question</th><th>Answer/Indicative content</th><th>Marks</th><th>Guidance</th></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td>not given both mark parts.</td></tr>
<tr><td></td><td></td><td></td><td>Total</td><td>4</td><td></td></tr>
<tr><td>2</td><td>a</td><td></td><td>Mark in pairs, 1 mark for advantage, 1 mark for expansion.
e.g.
Easier / quicker to create…
…. Language closer to English / uses keywords
Provides higher level of abstraction from the underlying processor…
… don't have to (usually) deal with allocating memory
Portable // processor independent…
… can write once for many processor types (e.g. mobile)</td><td>4
(AO1 1b, AO2 1b)</td><td>Accept other reasonable advantages and descriptions</td></tr>
<tr><td></td><td>b</td><td></td><td>1 mark per row
<table><tr><th>Statement</th><th>Compiler</th><th>Interpreter</th><th>Both</th></tr><tr><td>Translates high-level code to low-level instructions.</td><td></td><td></td><td>✓</td></tr><tr><td>Produces an executable file.</td><td>✓</td><td></td><td></td></tr><tr><td>Program needs to be translated every time it is run.</td><td></td><td>✓</td><td></td></tr></table></td><td>3
(AO1 1a)</td><td></td></tr>
<tr><td></td><td></td><td></td><td>Total</td><td>7</td><td></td></tr>
</table>

Question			Answer/Indicative content	Marks	Guidance
3			1 mark per bullet, max 4. e.g. • Editor • …to enable **program code** to be entered/edited • Error diagnostics / debugging • …to display information about errors (syntax / run-time) / location of errors • … suggest solutions • Run-time environment • …to enable to the program to be run • … check for run time errors / test the program • Translator / compiler / interpreter • …to convert the high level code into <u>machine code</u> / <u>low level code / binary</u> • …to enable to code to be executed / run • Breakpoints • …to stop/pause program execution at a specific point • Watch window • …to check contents of variables • Stepping • …to execute program line by line • Syntax completion… • …suggests/corrects code • Keyword highlighting / colour coding keywords / pretty printing… • …colours command words / variables	4	One mark for identifying, one mark for describing. Accept description of a tool without (or with incorrect) naming of the tool. Allow sensible descriptions which go across pairs or name other tools sensibly (e.g. editor / highlighting syntax) Allow any sensible tool that an IDE provides (e.g. auto documentation, help tools, pretty printing etc.) <u>Examiner's Comments</u> Most candidates were able to gain at least some marks from this question, with a significant number able to access all 4 marks. A list of common tools and facilities available in an IDE is given in the specification (section 2.5 'Translators and facilities of languages') although other sensible tools that candidates may have had experience with were also accepted. Although many candidates were able to name the tools, they often struggled to describe these and give more comprehensive answers. Again, extensive practical use of a high-level language programming language alongside an IDE would have been extremely beneficial to candidates and centres.
			Total	4	